Week 7 Lab Report Louis Desgrée: Scanning the Web Server with Nikto

I began this assignment by ensuring that my attacker machine, originally created in Week 4, was configured to reach the vulnerable web server I had set up on VMnet2. In VirtualBox, I added a second network adapter on VMnet2 to the attacker machine so it could properly communicate with the target system. After confirming that the attacker machine could successfully ping and access the server's IP address, I moved on to installing and using Nikto as my chosen vulnerability scanner.

Nikto is an open-source tool specifically designed to scan web servers for common misconfigurations, insecure files, and known vulnerabilities. I selected it because it is both easy to install and particularly effective at discovering weaknesses in web-based applications. Its command-line interface and minimal configuration requirements make it a straightforward choice when testing environments such as a deliberately vulnerable application. Another benefit is its continuously updated vulnerability database, which helps ensure that newly discovered issues are included in the scan.

In order to install Nikto on my attacker machine, I first opened a terminal and updated my system by running the standard commands for Kali Linux. I used "sudo apt update" and then "sudo apt upgrade -y" to make sure that all repositories and installed packages were current. Once that process finished, I installed Nikto by typing "sudo apt install nikto -y." This single command retrieved the Nikto package from the Kali repositories and set it up on my machine without further configuration. After installation, I verified that Nikto was working by typing "nikto -Version," which returned the tool's version number and confirmed it was ready to use.

When running the actual scan, I used the simplest configuration to capture a broad range of potential vulnerabilities on the target web server. Because my server was accessible at 127.0.0.1 in the attacker machine's environment, I entered the command "nikto -h 127.0.0.1 -port 80." I specified the port because Apache was listening on port 80, which I had previously confirmed by checking the netstat output. This basic setup allowed Nikto to automatically probe for common issues, including missing security headers, directory indexing, and potential backdoors. I chose these settings because Nikto already includes numerous tests in its default scan profile, which makes it ideal for an initial sweep of a vulnerable web application.

```
┌──(louis㉿kali)-[~]
└─$ nikto -h 127.0.0.1 -port 80

- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          127.0.0.1
+ Target Hostname:    127.0.0.1
+ Target Port:        80
+ Start Time:         2025-02-23 16:27:33 (GMT-8)
---------------------------------------------------------------------------
```

After the scan finished, I reviewed the report that Nikto generated. It indicated several noteworthy findings, such as the absence of the X-Frame-Options header, which can enable

clickjacking attacks, and a lack of the X-Content-Type-Options header, which can lead to MIME-type sniffing issues. It also flagged potential backdoor file managers, exposed server-status pages, and other files that might reveal sensitive information. These findings confirmed that the target web server was configured insecurely, which aligned with the objective of scanning a deliberately vulnerable setup. Below is a condensed excerpt of the scan output to illustrate the types of vulnerabilities identified:

```
┌──(louis㉿kali)-[~]
└─$ nikto -h 127.0.0.1 -port 80

- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          127.0.0.1
+ Target Hostname:    127.0.0.1
+ Target Port:        80
+ Start Time:         2025-02-23 16:25:03 (GMT-8)
---------------------------------------------------------------------------
+ Server: Apache/2.4.63 (Debian)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://d
eveloper.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user age
nt to render the content of the site in a different fashion to the MIME type. Se
e: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-
content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /: Server may leak inodes via ETags, header found with file /, inode: 29cf, si
ze: 62bb8ebc5a834, mtime: gzip. See: http://cve.mitre.org/cgi-bin/cvename.cgi?na
me=CVE-2003-1418
```

```
+ /DVWA/.gitignore: .gitignore file found. It is possible to grasp the directory
 structure.
+ /DVWA/wp-content/themes/twentyeleven/images/headers/server.php?filesrc=/etc/ho
sts: A PHP backdoor file manager was found.
+ /DVWA/wordpress/wp-content/themes/twentyeleven/images/headers/server.php?files
rc=/etc/hosts: A PHP backdoor file manager was found.
+ /DVWA/wp-includes/Requests/Utility/content-post.php?filesrc=/etc/hosts: A PHP
backdoor file manager was found.
+ /DVWA/wordpress/wp-includes/Requests/Utility/content-post.php?filesrc=/etc/hos
ts: A PHP backdoor file manager was found.
+ /DVWA/wp-includes/js/tinymce/themes/modern/Meuhy.php?filesrc=/etc/hosts: A PHP
 backdoor file manager was found.
+ /DVWA/wordpress/wp-includes/js/tinymce/themes/modern/Meuhy.php?filesrc=/etc/ho
sts: A PHP backdoor file manager was found.
+ /DVWA/assets/mobirise/css/meta.php?filesrc=: A PHP backdoor file manager was f
ound.
+ /DVWA/login.cgi?cli=aa%20aa%27cat%20/etc/hosts: Some D-Link router remote comm
and execution.
+ /DVWA/shell?cat+/etc/hosts: A backdoor was identified.
+ /DVWA/.dockerignore: .dockerignore file found. It may be possible to grasp the
 directory structure and learn more about the site.
+ 8074 requests: 0 error(s) and 26 item(s) reported on remote host
+ End Time:           2025-02-23 16:28:34 (GMT-8) (5 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested


      ******************************************************************
      Portions of the server's headers (Apache/2.4.63) are not in
      the Nikto 2.5.0 database or are newer than the known string. Would you lik
e
      to submit this information (*no server specific data*) to CIRT.net
      for a Nikto update (or you may email to sullo@cirt.net) (y/n)? n
```

This excerpt demonstrates that Nikto was able to detect multiple issues on the server, including missing headers and open directories. These vulnerabilities would be addressed in a production environment by restricting access, updating configurations, and removing any unnecessary files or scripts. In a testing environment such as this, they provide an excellent learning opportunity for understanding how attackers might discover and exploit weaknesses in web-based services.

In conclusion, by adding a VMnet2 network adapter to my attacker machine, installing Nikto, and running a simple scan command, I was able to generate a comprehensive overview of

the vulnerabilities present on my deliberately insecure web server. The minimal configuration and default settings were sufficient for Nikto to identify numerous issues that, in a real-world context, would demand immediate attention. This fulfills the requirements of the assignment by describing the scanner, explaining my configuration choices, outlining the steps taken to install the tool, and providing a clear scan report that highlights key vulnerabilities on the target system.